

Tytuł: Thursday Night Fever

Autorzy: **Gabriela Czarnicka (GC),**
Michał Cedro (MC)

Ostatnia modyfikacja: 15.06.2026

Spis treści

1. Repozytorium git	1
2. Wstęp	1
3. Specyfikacja	1
3.1. Opis ogólny algorytmu	1
3.2. Tabela zdarzeń	2
4. Architektura	2
4.1. Moduł: top	2
4.1.1. Schemat blokowy	2
4.1.2. Porty	3
a) mou – mouse_ctl, input	3
b) vga – vga_ctl, output	3
4.1.3. Interfejsy	3
a) m2c – mouse_ctl to core	3
4.2. Rozprowadzenie sygnału zegara	3
5. Implementacja	4
5.1. Lista zignorowanych ostrzeżeń Vivado.	4
5.2. Wykorzystanie zasobów	4
5.3. Marginesy czasowe	4
6. Film.	4

1. Repozytorium git

Adres repozytorium GITa:

https://github.com/MichalCedro1/Thursday_night_fever.git

Link do prezentacji:

<https://canva.link/136bpvr8ow9sz3e>

2. Wstęp

W erze hiperrealistycznej grafiki i nieskończenie złożonych symulacji, nasz projekt stanowi swoisty wehikuł czasu, zabierający graczy w neonową, tętniącą życiem podróż do złotej ery lat siedemdziesiątych. Bezpośrednią inspiracją, która rozpałała w nas wizję tego przedsięwzięcia, był kultowy film „Saturday Night Fever” – to właśnie jego niezapomniana estetyka, taneczna gorączka i niepodrabialny magnetyzm stały się fundamentem dla naszej własnej, pikselowej interpretacji tamtych lat. Pomysł na Thursday Night Fever wyrósł z głębokiego pragnienia wskrzeszenia unikalnej magii klasycznych produkcji i nasycenia ich pulsującym rytmem disco. Naszym nadrzędnym celem było stworzenie dzieła w czystej estetyce retro, które

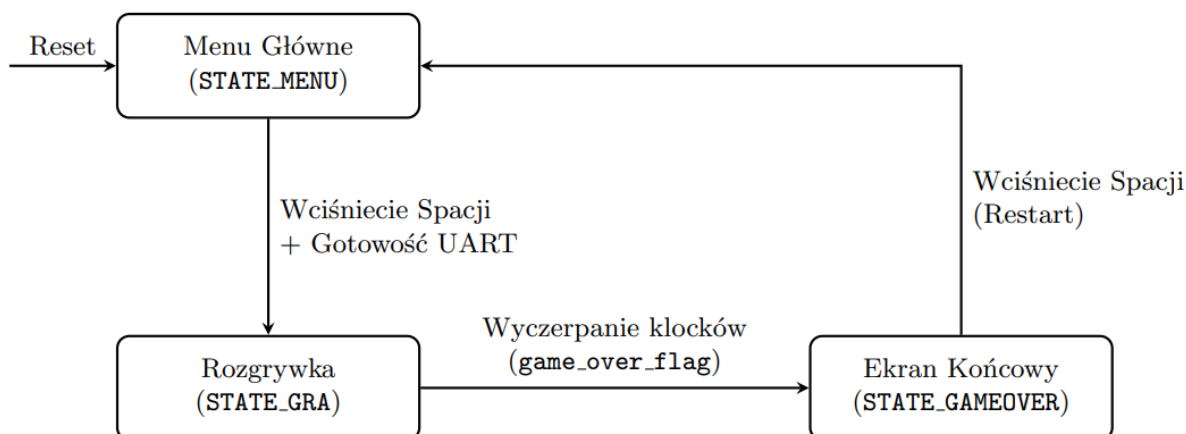
w starych wyjadaczach obudzi głęboką nostalgię, wyciskając łzę wzruszenia na wspomnienie godzin spędzonych w zadymionych salonach gier, a na nowe pokolenie rzuci urok, rozpalając w nim szczerą miłość do retro-kunsztu.

Jednak tym, co stanowi o prawdziwej duszy naszego projektu, jest muzyka. To właśnie wokół porywającej ścieżki dźwiękowej osnuliśmy dynamiczny, zręcznościowy trzon rozgrywki. W Thursday Night Fever obraz i dźwięk splatają się w jedno – każdy ruch i każda akcja dyktowane są przez nieubłagany beat. Tworząc tę grę, przyświecała nam jedna, fundamentalna myśl: zrealizować system, który sprawi, że każdy, kto stanie przed ekranem, poczuje nieodpartą potrzebę poddania się rytmowi. Nasz projekt to nie tylko gra zręcznościowa zaimplementowana na układzie FPGA; to wirtualna kula dyskotekowa, która ma za zadanie porwać wszystkich do tańca.

3. Specyfikacja

3.1. Opis ogólny algorytmu

Uproszczony schemat blokowy działania implementowanego algorytmu. Co się dzieje po starcie, jak wygląda przebieg działania, kiedy i pod jakimi warunkami się kończy.



Ewentualnie przykładowe screen-shoty tego, co w przybliżeniu chcielibyśmy uzyskać.

3.2. Tabela zdarzeń

Opis zdarzeń występujących podczas działania programu/urządzenia, zarówno zewnętrznych (interakcje z użytkownikiem), jak i wewnętrznych (specyficzne stany w algorytmie). Zdarzenia podzielone są na kategorie dotyczące różnych stanów działania programu. Kategorie powinny odpowiadać stanom ze schematu z pkt. 2.1.

Zdarzenie	Kategoria	Reakcja systemu
Zmiana stanu przełącznika SW0	Ekran startowy/Gra	Zmiana tekstury (wyglądu) gracza (Gabrysia / Michał)
Naciśnięcie klawisza Spacja	Ekran startowy	Wysłanie sygnału gotowości (0xD1) przez UART i oczekiwanie na przeciwnika
Odebranie sygnału gotowości (UART)	Ekran startowy	Uruchomienie gry (przejście do stanu Gra), wyzerowanie punktów i zegarów
Naciśnięcie klawisza A	Gra	Przesunięcie gracza w lewo, zmiana wyglądu na dopasowany do lewej strony
Naciśnięcie klawisza D	Gra	Przesunięcie gracza w prawo, zmiana koloru łapacza na dopasowany do prawej strony

Naciśnięcie Spacji w strefie "Perfect"	Gra	Usunięcie bloku, dodanie 2 pkt, ustawienie krótkiego opóźnienia na kolejne uderzenie
Naciśnięcie Spacji w strefie "Good"	Gra	Usunięcie bloku, dodanie 1 pkt, ustawienie krótkiego opóźnienia na kolejne uderzenie
Naciśnięcie Spacji poza strefą	Gra	Odjęcie 1 pkt, nałożenie długiego opóźnienia
Odpowiedni czas gry (zgodny z <i>beatmapą</i>)	Gra	Wygenerowanie nowego spadającego bloku o żadanym kolorze i prędkości
Zmiana własnego wyniku punktowego	Gra	Wysłanie zaktualizowanego wyniku (setki, dziesiątki, jednostki) do drugiego gracza przez UART
Odebranie danych o wyniku przez UART	Gra	Aktualizacja wyświetlanego na ekranie wyniku przeciwnika
Przetworzenie całej <i>beatmapy</i>	Gra	Zatrzymanie generowania klocków, wysłanie sygnału końca przez UART i oczekiwanie na ukończenie utworu przez przeciwnika.
Odebranie sygnału końca gry od przeciwnika	Gra	Ustawienie flagi zsynchronizowanego końca gry (<code>synced_game_over</code>) i przejście do Ekranu Końcowego.
Odebranie ostatecznego wyniku przeciwnika	Ekran końcowy	Porównanie wyników, wyłonienie zwycięzcy i zmiana koloru własnego wyniku (Zielony - wygrana, Czerwony - przegrana, Żółty - remis)
Naciśnięcie Spacji (przy gotowości obu graczy)	Ekran końcowy	Zapisanie wyniku do rankingu Highscore, powrót do ekranu startowego

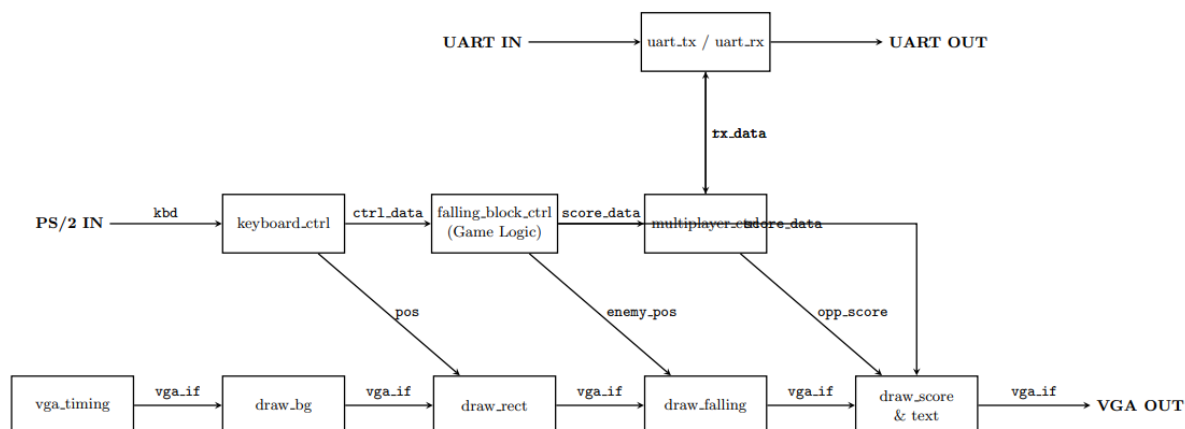
4. Architektura

Modułem najwyższego poziomu w strukturze projektu jest `top_vga_basys3`. Pełni on funkcję strukturalnego "opakowania" (top-level wrapper) spinającego wygenerowane rdzenie IP (generator zegara `clk_wiz_0`), sprzętowe bufor i porty wejścia/wyjścia z głównym modułem funkcjonalnym projektu (`top_vga`), w którym realizowana jest właściwa logika gry oraz z modułami sterującymi dźwiękiem.

4.1. Moduł: top

Osoba odpowiedzialna: GC/MC

4.1.1. Schemat blokowy



Miejsce na schemat blokowy modułu głównego

Uwaga:

- interfejsy dwukierunkowe rozbijamy na 2 interfejsy jednokierunkowe
- nazwa interfejsu stanowi prefiks nazwy sygnałów składowych
- w interfejsach nie uwzględniamy sygnałów globalnych (np. clk i rst).

4.1.2. Porty

a) sys – system signals, input

nazwa portu	opis
clk	Systemowy sygnał zegarowy z płytki
btnC	Przycisk centralny pełniący funkcję asynchronicznego resetu
sw0	Przełącznik konfiguracyjny (wybór postaci)

b) kbd – ps2 keyboard, input

nazwa portu	opis
PS2Clk	Linia zegarowa sprzętowego interfejsu klawiatury PS/2
PS2Data	Linia przesyłu danych interfejsu klawiatury PS/2

c) vga – vga ctl, output

nazwa portu	opis
Vsync	Sygnał synchronizacji pionowej VGA
Hsync	Sygnał synchronizacji poziomej VGA
vgaRed[3:0]	4-bitowa składowa czerwona koloru piksela
vgaGreen[3:0]	4-bitowa składowa zielona koloru piksela
vgaBlue[3:0]	4-bitowa składowa niebieska koloru piksela

d) *uart_rx – uart_receive, input*

nazwa portu	opis
uart_rx_pin	Linia odbierania danych szeregowych UART (od drugiego gracza)

e) *uart_tx – uart_transmit, output*

nazwa portu	opis
uart_tx_pin	Linia nadawania danych szeregowych UART (do drugiego gracza)

f) *aud – audio_out, output*

nazwa portu	opis
speaker	Sygnał PWM generujący dźwięk gry (mikser dwóch kanałów)
amp_en	Sygnał zezwolenia (enable) dla zewnętrznego wzmacniacza audio

g) *dbg – debug, output*

nazwa portu	opis
JA1	Wyjście diagnostyczne na złącze Pmod (lustrzane odbicie zegara pclk_mirror)

4.1.3. Interfejsy**a) *vga_if – interfejs danych wizyjnych VGA (przesył między modułami graficznymi)***

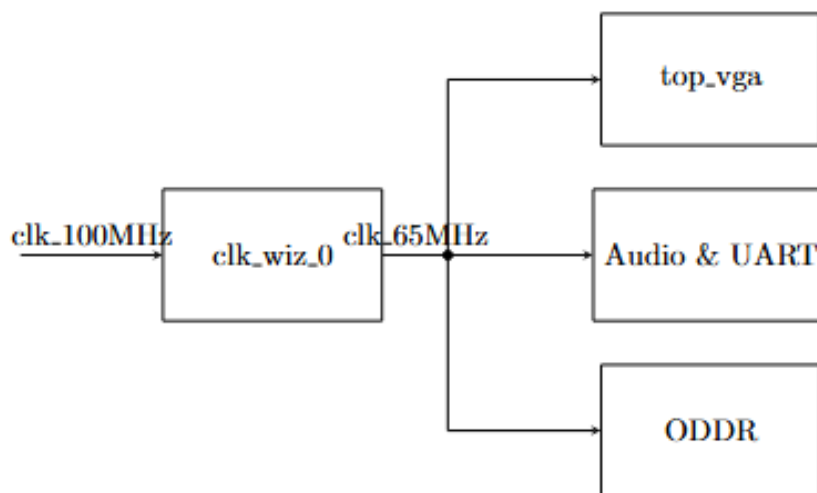
nazwa sygnału	opis
vcount[10:0]	Pionowa pozycja aktualnie rysowanego piksela na ekranie
hcount[10:0]	Pozioma pozycja aktualnie rysowanego piksela na ekranie
vsync	Sygnał synchronizacji pionowej
hsync	Sygnał synchronizacji poziomej
vblnk	Sygnał wygaszania pionowego (poza widocznym obszarem rysowania)
hblnk	Sygnał wygaszania poziomego (poza widocznym obszarem rysowania)
rgb[11:0]	12-bitowy kolor aktualnie rysowanego piksela

4.2. Rozprowadzenie sygnału zegara

Osoba odpowiedzialna: MC

Źródłem głównego sygnału zegarowego dla całego układu jest oscylator kwarcowy znajdujący się na płycie Basys3, dostarczający sygnał o częstotliwości 100 MHz. Sygnał ten jest wprowadzany do układu poprzez port wejściowy clk.

Aby zapewnić optymalne parametry czasowe dla wyświetlania obrazu w standardzie VGA (rozdzielczość 1024×768 przy 60 fps), modułem generatora zegara jest rdzeń IP Vivado (clk_wiz_0). Przetwarza on wejściowy sygnał 100 MHz na wyjściowy sygnał clk 65MHz o częstotliwości 65 MHz. Sygnał 65 MHz stanowi jedną, globalną domenę zegarową dla wszystkich modułów funkcjonalnych w projekcie. Został on doprowadzony do modułu głównego logiki (top_vga), modułów sterujących dźwiękiem (music_controller, tone_generator, audio_pwm), układów komunikacji (uart_tx, uart_rx) oraz modułu bufora wyjściowego (ODDR), który służy do celów diagnostycznych. Zastosowanie jednej częstotliwości dla całej logiki skutecznie zapobiega problemom z synchronizacją między modułami.



4.3. UART

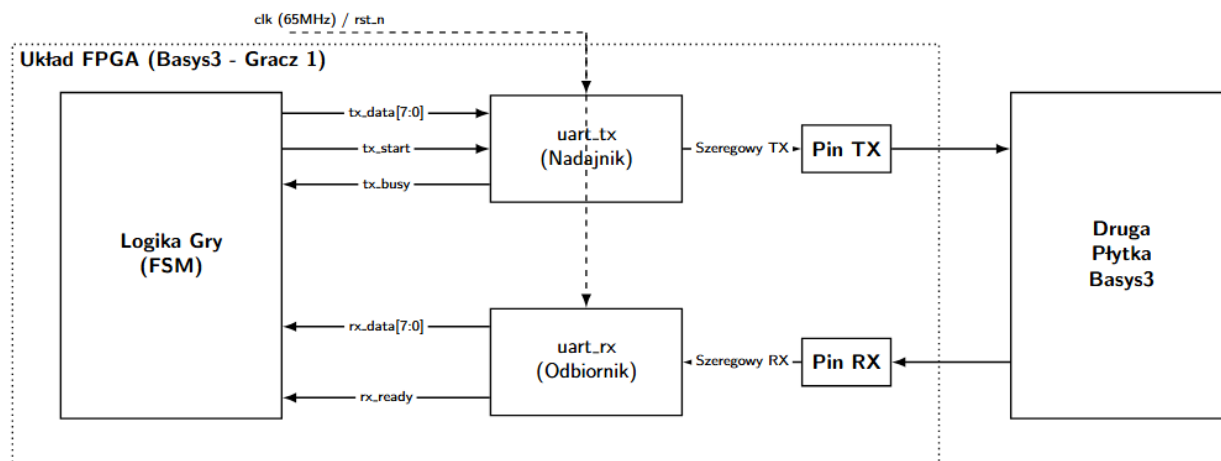
Osoba odpowiedzialna: MC

Podsystem komunikacji UART został zaimplementowany w celu umożliwienia dwukierunkowej wymiany danych między dwiema niezależnymi płytkami Basys3. Jest to fundament trybu wieloosobowego (multiplayer), pozwalający na bieżąco synchronizować stan gry (np. wybór postaci, pozycje obiektów, wyniki). Moduły zostały zaprojektowane do pracy w standardzie 8N1 (8 bitów danych, brak bitu parzystości, 1 bit stopu) z prędkością transmisji wynoszącą 115200 bps (Baud Rate), przy częstotliwości zegara systemowego 65 MHz.

Architektura podsystemu komunikacyjnego opiera się na dwóch głównych, niezależnych od siebie modułach sprzętowych zarządzanych przez maszyny stanów:

- **uart_tx (Nadajnik):** Moduł ten odpowiada za konwersję równoległych, 8-bitowych danych na szeregowy strumień bitów. Proces wysłania inicjowany jest pojedynczym impulsem na wejściu tx_start. Maszyna stanów przechodzi z trybu IDLE do stanu START, generując logiczne zero na linii wyjściowej tx. Następnie w stanie DATA przesuwając i wystawia kolejne bity informacji z częstotliwością zdefiniowaną przez parametr BIT_TICK (liczba taktów zegara przypadająca na jeden bit). Na zakończenie generowany jest bit STOP (logiczna jedynka). Podczas trwania całej operacji moduł utrzymuje wysoki stan na wyjściu tx_busy, co chroni układ sterujący przed nadpisaniem danych w trakcie ich wysłania.
- **uart_rx (Odbiornik):** Moduł odpowiedzialny za nasłuchiwanie i dekodowanie sygnału z zewnątrz. Co kluczowe, sygnał wejściowy rx jest asynchroniczny względem zegara systemowego. Aby uniknąć zjawiska metastabilności, zastosowano dwustopniowy synchronizator sprzętowy (rejstry rx_sync1 i rx_sync2). Odbiornik wykrywa opadające zbocze oznaczające bit startu, a następnie odczekuje połowę czasu trwania bitu (HALF_TICK). Dzięki temu próbkowanie kolejnych bitów danych następuje dokładnie w środku ich trwania, co znacząco zwiększa odporność systemu na szumy i zniekształcenia sygnału. Po poprawnym odebraniu całego bajtu danych i weryfikacji bitu stopu, moduł generuje impuls rx_ready, sygnalizując głównej logice, że na wyjściu rx_data znajduje się prawidłowa, nowa ramka informacji.

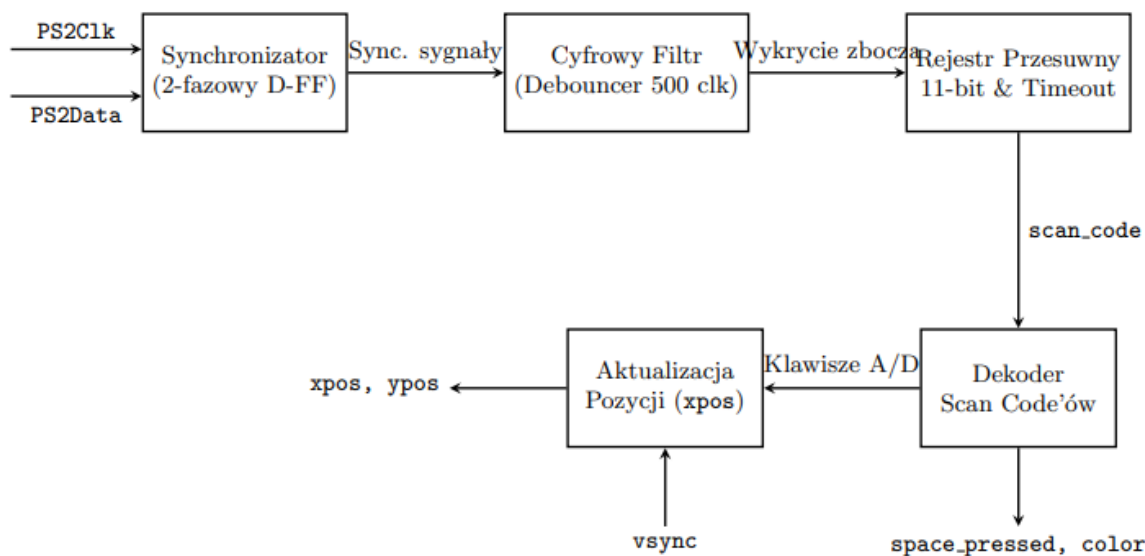
Sygnały wychodzące (tx) i wchodzące (rx) są bezpośrednio zmapowane na fizyczne piny złącz rozszerzeń na płycie FPGA. Pomiędzy dwoma połączonymi układami zastosowano połączenie krzyżowe – wyjście nadajnika pierwszej płytki trafia do wejścia odbiornika drugiej płytki i odwrotnie, co zapewnia bezkolizyjną komunikację w pełnym duplexie.



4.4. Keyboard

Osoba odpowiedzialna: MC

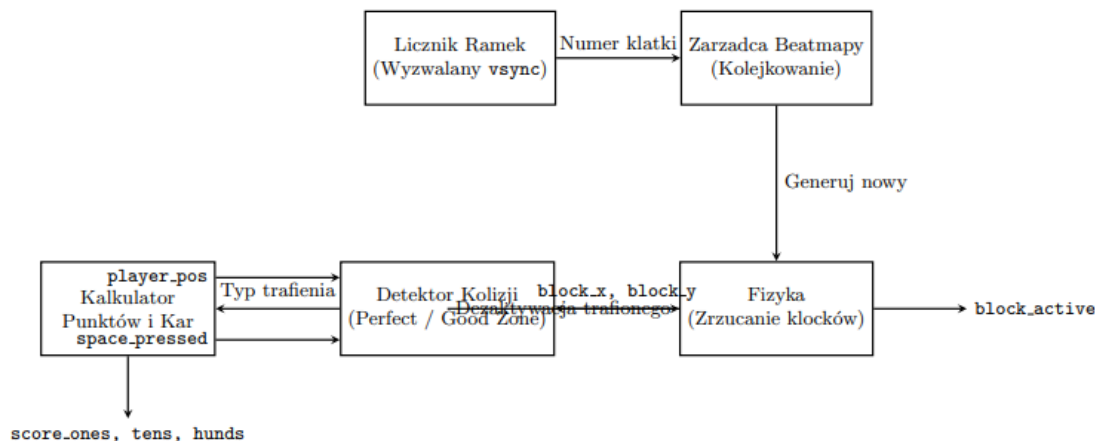
Moduł ten odpowiada za sprzętową obsługę klawiatury PS/2. Zawiera wielostopniową filtrację sygnału zapobiegającą stanowi metastabilnemu, debouncer zliczający próbki oraz maszynę przesuwającą (shift register) dekodującą tzw. scan codes.



4.5. Architektura modułu: falling_block_ctrl

Osoba odpowiedzialna: MC/GC

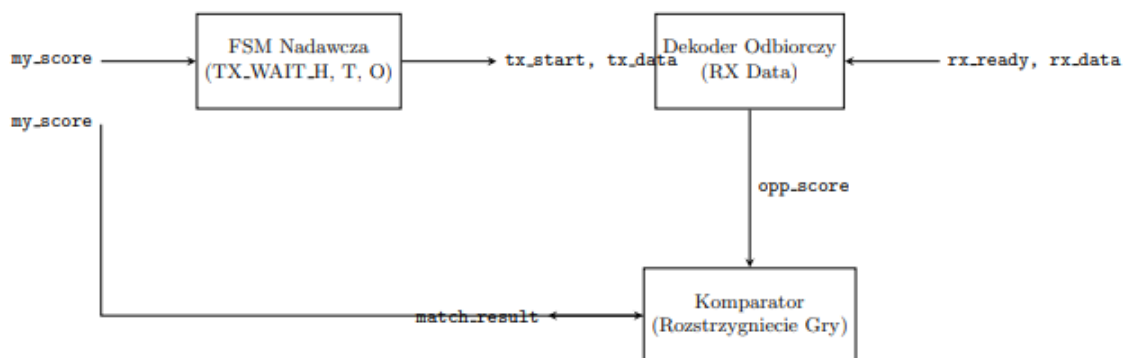
Serce logiki gry. Moduł korzysta z zaszytej "beatmapy" (ROM logiczny) do spawnowania nowych klocków bazując na globalnym liczniku klatek zsynchronizowanym z sygnałem vsync. Posiada również zaawansowany detektor kolizji dzielący trafienia na strefy (Perfect/Good/Miss).



4.6. Architektura modułu: multiplayer_ctrl

Osoba odpowiedzialna: MC

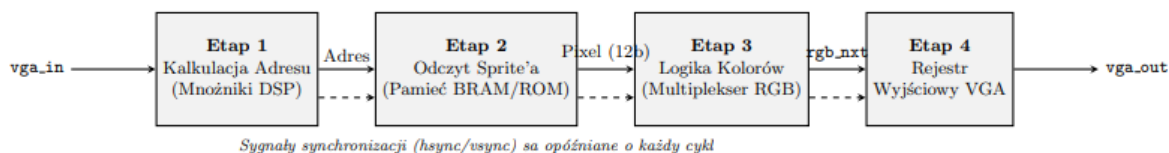
Moduł ten jest mostem łączącym logikę gry z układowymi transmisjami szeregową UART. Zawiera dedykowaną maszynę stanów wysyłającą trzy bajtowe ramki z wynikiem (Setki, Dziesiątki, Jedności) oraz dekodery odbierający statystyki od przeciwnika.



4.7. Potok renderingu grafiki (VGA Display Pipeline)

Osoba odpowiedzialna: MC/GC

Wszystkie zaawansowane moduły generujące obraz (drawrect, draw falling block) opierają się na czterostopniowym potoku (pipeline), co pozwala na osiągnięcie wysokich częstotliwości taktowania (spełnienie marginesów czasowych WNS) poprzez odciążenie ścieżki krytycznej.



4.8. Audio

Osoba odpowiedzialna: GC

Celem zaprojektowanego podsystemu audio jest sprzętowa generacja i odtwarzanie wielokanałowego dźwięku przy użyciu układu FPGA na płycie Basys3. Sygnał dźwiękowy wyprowadzany jest do za pomocą cyfrowo-analogowego wzmacniacza audio PmodAMP2.

Aby zapewnić pełne brzmienie, system został zrealizowany w architekturze dwukanałowej. Pozwala to na jednocześnie, równoległe odtwarzanie dwóch niezależnych ścieżek dźwiękowych – głównej melodii oraz linii basu. Podsystem operuje na sygnale zegarowym o częstotliwości 65 MHz i jest w pełni zintegrowany z głównym modułem układu.

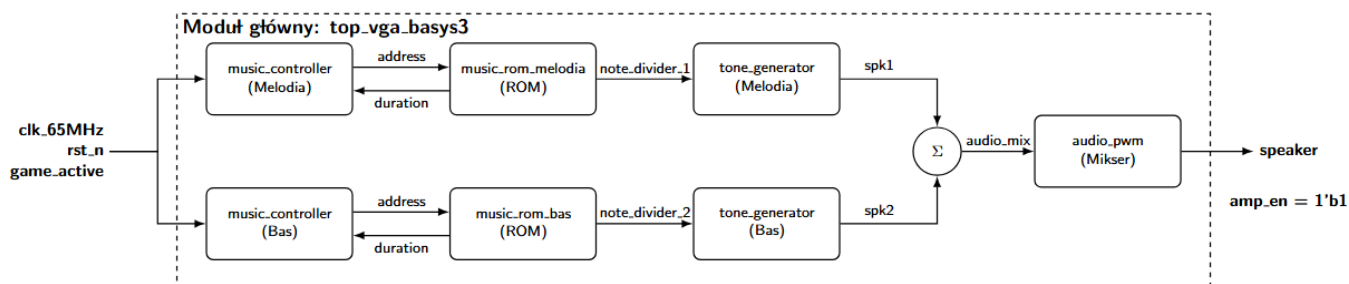
Każda ze ścieżek dźwiękowych (melodia i bas) posiada własny, dedykowany zestaw trzech modułów:

- **music_controller (Moduł sterujący):** Jest to sekwencer odpowiedzialny za odmierzanie czasu i zmianę nut w odpowiednim momencie. Moduł odbiera z pamięci informację o czasie trwania obecnej nuty (duration) i uruchamia wewnętrzny licznik taktów zegara. Gdy odliczony czas zrówna się z oczekiwanym czasem trwania nuty, kontroler zeruje timer i inkrementuje adres (address), przechodząc do kolejnego dźwięku. Posiada również wejście enable, które pozwala na pauzowanie lub wyłączanie muzyki w zależności od stanu gry.
- **music_rom_... (Pamięć asynchroniczna ROM):** Moduł ten pełni rolę partytury dla danego kanału (np. music_rom_melodia, music_rom_bas). Na podstawie otrzymanego od kontrolera adresu, moduł wystawia na wyjścia dwa 32-bitowe sygnały: note_divider (wartość dzielnika zegara definiująca wysokość dźwięku) oraz wspomniany wyżej duration.
- **tone_generator (Generator tonu):** Odpowiada za fizyczną generację fali akustycznej. Moduł przyjmuje sygnał note_divider. Jeśli wartość dzielnika wynosi 0, na wyjściu utrzymywany jest stan niski. W przeciwnym razie wewnętrzny licznik odmierza podaną liczbę taktów zegara głównego, a po jej osiągnięciu odwraca stan wyjścia cyfrowego. W ten sposób powstaje sygnał prostokątny o ściśle określonej częstotliwości akustycznej.

Ponieważ układ PmodAMP2 przyjmuje zsumowany sygnał, konieczne było połączenie obu wygenerowanych kanałów. Cyfrowe fale prostokątne z generatora melodii (spk1) oraz basu (spk2) są logicznie sumowane, tworząc wspólny, 3-bitowy sygnał magistrali.

Następnie połączony sygnał trafia do modułu audio_pwm. Jego zadaniem jest konwersja wartości zsumowanych przebiegów prostokątnych na ostateczny sygnał PWM (Pulse-Width Modulation). Sygnał ten jest podawany bezpośrednio na złącze GPIO płytki (speaker).

Aby zasilanie i wzmacnienie sygnału w zewnętrznym module PmodAMP2 mogło funkcjonować prawidłowo, konieczne było również przypisanie stałej logicznej jedynki (1'b1) do dedykowanego wyjścia sterującego amp_en.



5. Implementacja

5.1. Lista zignorowanych ostrzeżeń Vivado.

Identyfikator ostrzeżenia	Liczba wystąpień	Uzasadnienie
[Synth 8-3917]	1	Port amp_en został celowo na stałe ustawiony na stan wysoki, aby wzmacniacz audio był zawsze aktywny. Ostrzeżenie ma charakter wyłącznie informacyjny.
[Synth 8-7129]	14	Moduł draw_bg nie wykorzystuje części swoich wejść (vga_in.rgb, state), ponieważ generuje tło niezależnie od danych pochodzących z wcześniejszych etapów potoku graficznego. Jest to zamierzone działanie.
[Synth 8-7080]	1	Narzędzie nie zastosowało równoległej syntezy wielowątkowej. Komunikat dotyczy konfiguracji środowiska Vivado i nie ma wpływu na działanie projektu.
[Synth 8-5835]	1	Zapotrzebowanie na pamięć przekroczyło liczbę dostępnych bloków BRAM. Vivado automatycznie wykorzystało dodatkowo pamięć rozproszoną (LUT-RAM), dzięki czemu projekt został poprawnie zaimplementowany.
[Synth 8-3936]	1	Wykryto nieużywany rejestr w logice modułu draw_falling_block. Narzędzie automatycznie usunęło go podczas optymalizacji.
[DRC DPIP-1]	4	Bloki DSP nie posiadają pełnego potokowania na wejściach. Pomimo tego projekt spełnia wszystkie wymagania czasowe dla częstotliwości 65 MHz.
[DRC DPOP-1], [DRC DPOP-2]	4	Bloki DSP nie posiadają pełnego potokowania na wyjściach. Dodatkowe rejestry nie były konieczne, ponieważ uzyskano poprawne wyniki analizy czasowej.

5.2. Wykorzystanie zasobów

Resource	Utilization	Available	Utilization %
LUT	4840	20800	23.27
LUTRAM	8	9600	0.08
FF	1467	41600	3.53
BRAM	48	50	96.00
DSP	2	90	2.22
IO	24	106	22.64
BUFG	2	32	6.25
MMCM	1	5	20.00

5.3. Marginesy czasowe

Timing	Setup Hold
Worst Negative Slack (WNS):	1.988 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	3601
Implemented Timing Report	

Timing	Setup Hold
Worst Hold Slack (WHS):	0.041 ns
Total Hold Slack (THS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	3601
Implemented Timing Report	

6. Konfiguracja sprzętu

